

Introduction aux modèles génératifs

Contenu

- Du modèle génératif au modèle discriminatif
- Exemple du modèle de mélange gaussien
- Chaînes de Markov
- Auto-encodeurs
- Espaces latents

[[Diapositives du panorama d'introduction](#)] et [[Diapositives du cours 1](#)]

Les modèles décisionnels considérés jusqu'ici sont des modèles *discriminatifs*. À partir d'une observation x , l'objectif de modèle est d'identifier, à partir des propriétés de x , la valeur de la variable à expliquer y . Autrement dit, un modèle discriminatif modélise la probabilité conditionnelle $P(Y|X)$ (la probabilité *a posteriori*) à partir des données observées.

Néanmoins, ce n'est pas la seule façon de construire un modèle décisionnel. On peut ainsi formuler la question de la façon suivante : « *quelle donnée représente le mieux chacune des classes d'intérêt ?* ». Ceci revient à vouloir décrire chacune des classes d'intérêt en fonction des observations. Autrement dit, le modèle génératif s'intéresse à la probabilité conditionnelle $P(X|Y)$ (la *vraisemblance*).

Cette formulation est à rapprocher des modèles d'"[estimation de densité](#)" où l'on cherche à décrire la distribution des données, sans se préoccuper d'une quelconque notion de classe. Un modèle d'estimation de densité s'intéresse à décrire $P(X)$, ce qui peut être interprété comme un modèle génératif avec une unique classe d'intérêt.

Le plus souvent, on cherchera une fonction $g(z) \in \mathbb{R}^d$ qui produit des échantillons $\hat{x}$ similaires aux observations réelles $x_1, \dots, x_n$. Pour faciliter la modélisation, on exigera que la variable z dite *latente* suive une loi de probabilité connue (souvent normale ou uniforme) : $p(\hat{g}(z)) \approx p(x)$ avec $z \sim \mathcal{N}(0, I)$.

Du modèle génératif au modèle discriminatif

L'utilisation des modèles génératifs présente plusieurs intérêts :

- échantillonnage: il est possible de produire de nouvelles observations en échantillonnant $P(Y|X)$,
- extension à plusieurs classes: si l'on connaît $P(Y = y|X)$ pour $y \in \{y_1, \dots, y_n\}$, il suffit de connaître pour $P(Y = y_{n+1}|X)$ pour étendre le modèle.
- estimation de la confiance d'une prédiction.

Pour ce troisième point, il est en effet possible de passer d'un modèle génératif à un modèle discriminatif très facilement à l'aide du théorème de Bayes. Considérons une variable aléatoire explicative X et une variable aléatoire à expliquer Y . Le modèle génératif est représenté par la loi de probabilité jointe $P(X|Y)$.

On suppose connaître l'a priori bayésien sur la distribution statistique $P(Y)$ (par exemple, pour des valeurs nominales, on connaît la proportion de chacune des classes). Par le théorème de Bayes :

$$P(Y = y|X = x) = \frac{P(X = x|Y = y) \cdot P(Y = y)}{P(X = x)}$$

Bien sûr, on ne connaît pas $P(X)$ mais ce n'est pas important ! Si l'on souhaite trouver la valeur de y la plus vraisemblable pour l'observation x , on cherche :

$$\arg \max_y P(Y = y|X = x) = \arg \max_y \frac{P(X = x|Y = y) \cdot P(Y = y)}{P(X = x)} = \arg \max_y P(X =$$



et $P(X)$ n'intervient pas dans l' $\arg \max_y$. On peut donc transformer notre modèle génératif en classifieur sans grande difficulté.

Une image couramment employée pour distinguer un classifieur qui utilise un modèle discriminatif et un autre qui utilise un modèle génératif est la suivante :

Alice et Bob se rendent dans un zoo où se trouvent seulement deux espèces : des lions et des éléphants. À la fin de leur visite, on leur propose un quiz. On montre aux deux personnes une image et on leur demande s'il s'agit d'un lion ou d'un éléphant.

Alice réfléchit et se fait un dessin mental d'un lion et un dessin d'un éléphant. Après un temps de réflexion, elle compare chaque dessin à l'image du quiz et choisit celle qui est

la plus similaire, avant de conclure : « c'est un lion ».

Bob réfléchit également en cherchant à se rappeler des différences entre les deux espèces. Constatant que l'image représente un animal à crinière et que les lions en ont une mais pas les éléphants, Bob conclut : « c'est un lion ».

Dans cette analogie, Alice est le *modèle génératif* et Bob est le *modèle discriminatif*.

Exemple du modèle de mélange gaussien

Le modèle de mélange gaussien est un bon exemple de modèle génératif. On rappelle que ce modèle cherche à approcher $P(X)$ par une somme de m gaussiennes (voir la [section à ce sujet de RCP208 pour des rappels](#)):

$$p_{\theta}(x) = \sum_{i=1}^m \alpha_i \mathcal{N}(\mu_i, \sigma_i)$$

avec:

- $\theta = \alpha_1, \dots, \alpha_m, \mu_1, \dots, \mu_m, \sigma_1, \dots, \sigma_m$ les paramètres du mélange,
- α_i le coefficient de pondération de la i -ème gaussienne,
- μ_i, σ_i le vecteur des moyennes et la matrice de variance-covariance de la i -ème gaussienne.

Implicitement, cela revient à considérer que les données sont réparties dans m classes d'intérêts, que l'on peut écrire par la formule des probabilités totales :

$$p_{\theta}(x) = \sum_{i=1}^m \alpha_i \mathcal{N}(\mu_i, \sigma_i) = \sum_{i=1}^m p_{\theta}(x|y=i)$$

où chaque classe est décrite par la composante correspondante du mélange.

On peut facilement échantillonner dans la variable aléatoire $\hat{X}$ dont la loi de probabilité est décrite par le mélange, mais aussi dans chaque composante du mélange, décrite par sa loi normale.

Comment maintenant utiliser le modèle de mélange gaussien comme classifieur ? La classe d'une donnée x s'obtient en cherchant la valeur de i pour laquelle $P(Y=i|X=x)$ est la plus élevée. Par théorème de Bayes :

$$\arg \max_i [\log P(Y = i | X = x)] = \arg \max_i \left[\log \frac{P(X = x | Y = i) \cdot P(Y = i)}{P(X = x)} \right]$$

Le dénominateur n'intervient pas dans l'"arg max", nous pouvons donc le retirer :

$$\begin{aligned} \arg \max_i [\log P(Y = i | X = x)] &= \arg \max_i [\log(P(X = x | Y = i)) + \log P(Y = i)] \\ &= \arg \max_i [\log f_i(x | \theta_i) + \log \alpha_i] \end{aligned}$$

où $f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp(-\frac{(x-\mu)^2}{2\sigma^2})$ est la densité de probabilité de la loi normale.

La classe de x est donc la composante du mélange pour laquelle la donnée est la plus (log-)vraisemblable, ce qui semble plutôt intuitif !

Chaînes de Markov

Un processus de Markov à temps discret est une séquence $(X_i)_{1 \leq i \leq \dots}$ où X est une variable aléatoire à valeurs dans un espace d'états E . On dit que X_n est l'état du processus à l'instant n . Lorsque l'espace d'état E a un cardinal fini (c'est-à-dire un nombre fini d'éléments), alors on parle de *chaîne de Markov*.

On définit la propriété de Markov comme l'indépendance entre l'état de la chaîne de Markov à l'instant $n + 1$ par rapport à ses états à l'instant $n - 1, n - 2, \dots$:

$$P(X_{n+1} = j | X_0 = i_0, X_1 = i_1, \dots, X_n = i) = P(X_{n+1} = j | X_n = i)$$

Autrement dit, les valeurs prises par la chaîne de Markov ne dépend que de ses valeurs à l'instant précédent. Cette propriété est parfois résumée de la façon suivante : « *la prédiction du futur ne nécessite pas de connaître le passé, seulement le présent* ».

Par la suite, on suppose que la chaîne de Markov respecte la condition d'"homogénéité", c'est-à-dire que la probabilité de passer d'un état i à partir d'un état j ne dépend pas du temps :

$$\forall n \geq 1, P(X_{n+1} = j | X_n = i) = P(X_n = j | X_{n-1} = i)$$

On note alors $p_{i,j} = P(X_1 = j | X_0 = i)$ la *probabilité de transition* de l'état i à l'état j .

Si le cardinal de l'espace d'états E est entier, alors on peut construire la matrice

$M = (p_{i,j})_{1 \leq i \leq n, 1 \leq j \leq n}$, que l'on appelle la *matrice de transition*.

Les chaînes de Markov peuvent être conceptualisées comme une forme de modèle génératif. Ces dernières sont très simples à entraîner et sont paramétrées par leurs probabilités de transition. À partir des séquences observées, on peut compter la fréquence de chaque transition ($i \rightarrow j$) et donc en déduire $p_{i,j}$. Il devient alors possible, en connaissant $X_0 = i$, de générer la séquence $X_1, X_2, \dots, X_n$ la plus probable en utilisant la règle suivante :

$$X_{n+1} = \arg \max_j P(X_{n+1} = j | X_n = i) = \arg \max_j p_{i,j}$$

Les chaînes de Markov présentées ci-dessus sont malheureusement peu expressives : elles ne peuvent pas modéliser des phénomènes dépendant de plus qu'un seul pas de temps. Néanmoins, on peut étendre leur définition pour prendre en compte un historique plus long. On parle de chaîne de Markov *d'ordre k* lorsque la prédiction du futur ne nécessite pas de connaître plus de k pas de temps dans le passé :

$$P(X_{n+1} = j | X_0 = i_0, \dots, X_n = i) = P(X_{n+1} = j | X_{n-k} = i_{n-k}, \dots, X_n = i)$$

Dans ce cas, la matrice de transition est multidimensionnelle et on cherche à estimer :

$$p_{i_1, i_2, \dots, i_k} = P(X_n = i_k | X_{n-1} = i_{k-1}, X_{n-2} = i_{k-2}, \dots, X_{n-k} = i_1)$$

La complexité en espace de la matrice de transition est malheureusement exponentielle selon k . Si le cardinal de l'espace d'états E est élevé, la matrice de transition complète peut devenir bien plus grande que ce que l'on peut stocker en mémoire. Si beaucoup de transitions sont impossibles (c'est-à-dire beaucoup de $p_{i,j} = 0$), on peut utiliser des matrices creuses pour économiser de la mémoire.

Note : cela revient à une chaîne de Markov d'ordre 1 dont l'espace d'états encode l'historique des états passés, c'est-à-dire à remplacer E par $E^k = \underbrace{E \times E \times \dots \times E}_{k \text{ fois}}$.

Auto-encodeurs

Un auto-encodeur est un type particulier de réseau de neurones artificiels. Cette architecture modélise une fonction $\mathcal{H}$ portant sur une variable aléatoire $X \in \mathbb{R}^n$ telle que:

$$\|\mathcal{H}(x) - x\| \leq \varepsilon$$

c'est-à-dire que l'image de x par $\mathcal{H}$ est une reconstruction de x à une erreur ε près.

En règle générale, un auto-encodeur est divisé en deux parties :

- un encodeur $\mathcal{E} : \mathbb{R}^n \rightarrow \mathbb{R}^d$;
- un décodeur $\mathcal{D} : \mathbb{R}^d \rightarrow \mathbb{R}^n$;

et $\mathcal{H}$ est l'application successive de l'encodage puis du décodage, c'est-à-dire :

$$\mathcal{H} = \mathcal{D} \circ \mathcal{E}$$

En principe, l'encodeur *compresse* l'information en réduisant la dimension de x . On a donc en général $d \leq n$: on essaie d'encoder x avec moins de variables que le vecteur original. Par la suite, on notera $z = \mathcal{E}(x)$ le *code* associé à x .

L'auto-encodeur réalise une tâche de **reconstruction**, on cherche donc les poids θ du réseau tels que :

$$\theta^* = \argmin_{\theta} \mathcal{L}(x, \hat{x}) = \|\mathcal{D}(\mathcal{E}(x)) - x\|$$

où $\mathcal{L}$ est une fonction de coût de régression, généralement une erreur quadratique moyenne ou une erreur absolue moyenne.

L'encodeur compresse donc x en réduisant sa dimension tandis que le décodeur reconstruit x à partir du code réduit z .

L'espace des codes réduits $\mathcal{Z} = \mathbb{R}^d$ est appelé *espace latent*. Le décodeur $\mathcal{D}$ est un modèle génératif $P(X|z)$. Échantillonner dans $\mathcal{Z}$ permet de produire des observations $x \in \mathbb{R}^n$.

Espaces latents

La notion d'espace latent est un élément important de la compréhension des modèles génératifs modernes. Bien souvent, on ne dispose pas d'une variable y d'intérêt précise qui permettrait la supervision. À la place, on cherche un espace sous-jacent qui permet de bien expliquer la structure des données. C'est cet espace que l'on appelle l'espace latent. Le principe est le suivant : des différences importants dans l'espace des observations peuvent être expliquées par de faibles variations dans l'espace latent du fait de régularités sous-jacentes de la distribution (par exemple, deux images de chien peuvent être très différentes dans la représentation matricielle mais sont sémantiquement proches).

Nous avons déjà vu de nombreux espaces latents : les plongements lexicaux (Word2Vec, ...), les cartes d'activation d'un CNN, le résultat d'une projection par t-SNE, mais aussi donc l'espace intermédiaire d'un auto-encodeur.

Lorsque l'on connaît une fonction $d : \mathcal{Z} \rightarrow \mathcal{X}$ qui permet de faire la correspondance entre un code de l'espace latent et une donnée de l'espace des observations, il est possible de manipuler les codes pour générer de nouvelles données.

Par exemple, pour deux codes z_1 et z_2 , il est possible d'interpoler dans l'espace latent pour reconstruire des mélanges en se promenant le long du segment $[z_1, z_2]$:

$$x_{\text{mélange}} = \mathcal{D}(\alpha \cdot z_1 + (1 - \alpha) \cdot z_2)$$

avec $\alpha \in [0, 1]$.

Dans le cas auto-encodeurs, nous disposons aussi de la fonction $e : \mathcal{X} \rightarrow \mathcal{Z}$ qui permet de plonger une donnée dans l'espace latent, ce qui est bien pratique pour comprendre la structure de l'espace latent ou explorer le voisinage d'une observation. Nous verrons plus tard que cela n'est pas toujours le cas.

Les auto-encodeurs présentés dans cette section ont toutefois quelques limites :

- pour échantillonner dans $\mathcal{Z}$ afin de générer de nouvelles données, il serait préférable de connaître la distribution statistique des points dans cet espace. Malheureusement, nous ne la connaissons pas a priori et, même si on peut l'estimer, en pratique cette distribution ne pas très lisse. En particulier, nous n'avons pas de garantie que les codes z se situant hors du voisinage des codes vus pendant l'apprentissage (et donc dans des zones à faible densité) aient du sens une fois décodés.
- l'encodeur n'est généralement pas injectif : deux observations différentes x_1 et x_2 peuvent avoir des projections quasiment égales $z_1 \approx z_2$.

Nous verrons également que le choix de la dimension d de l'espace latent n'est pas anodin et peut avoir un impact significatif sur les résultats.

Une façon de faciliter l'échantillonnage est de contraindre $p(z)$ à avoir une certaine forme. C'est la solution adoptée par les auto-encodeurs variationnels que nous verrons dans la suite du cours.